



# Bus Reservation System

Comprehensive Technical Feasibility Study & Implementation Roadmap for  
GitHub Packages Distribution

|                  |                                     |
|------------------|-------------------------------------|
| Project:         | Bus Reservation System              |
| Repository:      | Jashan-randhawa/Bus-Resrvation      |
| Author:          | Jashan Randhawa                     |
| Current Release: | v2.0.0 (Cloud & Dockerized)         |
| Registry Target: | GitHub Container Registry (ghcr.io) |
| Date Generated:  | October 2026                        |

This document outlines how the Bus Reservation System codebase can be packaged, distributed, and versioned on GitHub Packages to establish enterprise-grade software delivery and enable one-line deployments.

---

# 1. Executive Summary

Following the successful release of **v2.0.0**—which introduced containerization (Docker, PHP 8.1, Apache, MySQLi) and cloud deployment on Render with TiDB Cloud Serverless—the next strategic milestone is software distribution via **GitHub Packages**.

GitHub Packages is a package hosting service that allows developers to publish and consume packages alongside their source code. Integrating GitHub Packages into this repository provides several advantages:

### Seamless Distribution

Allows anyone to pull and spin up the complete bus reservation platform using standard container tooling without needing local compilation or PHP setup.

### Versioned Immutability

Every release tag (e.g., `v2.0.0` , `v2.1.0` ) produces a sealed, tamper-proof container artifact linked directly to its source commit.

**Key Finding:** Because the project now possesses a battle-tested, production-grade `Dockerfile` and is deployed live on Render, the **highest ROI, zero-friction package** to publish immediately is an **OCI Container Image to GitHub Container Registry (ghcr.io)**.

# 2. GitHub Packages Evaluation Matrix

GitHub Packages supports several distinct package ecosystems. Below is a comprehensive assessment of how each package format applies to the Bus Reservation System:

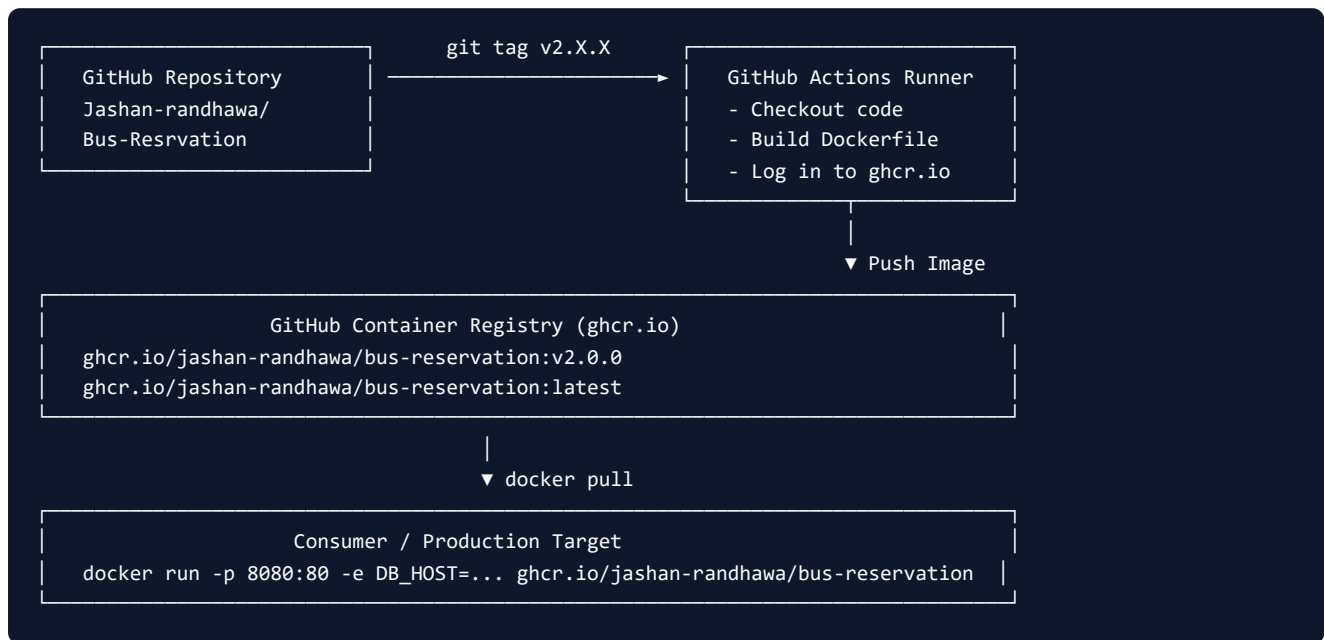
| Package Format         | Target Registry                           | Feasibility                      | Impact & Value                                                                                 | Implementation Effort                                                          | Recommendation     |
|------------------------|-------------------------------------------|----------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------|
| Docker / OCI Container | ghcr.io/jashan-randhawa/bus-reservation   | Immediate (100%)                 | High: Enables one-command run, Kubernetes deployment, and adds official GitHub Packages badge. | Low (Single GitHub Actions workflow file)                                      | RECOMMENDED #1     |
| Composer (PHP Library) | packagist.org or GitHub Composer Registry | Medium (Requires modularization) | Moderate: Extracts core booking & PNR algorithms into reusable PHP utility package.            | Medium (Requires creating <code>composer.json</code> & refactoring core logic) | PHASE 2 (OPTIONAL) |

| Package Format                  | Target Registry                              | Feasibility             | Impact & Value                                                                                       | Implementation Effort                                  | Recommendation          |
|---------------------------------|----------------------------------------------|-------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------|-------------------------|
| <b>npm (Frontend Component)</b> | GitHub npm Registry / <code>npmjs.com</code> | Medium                  | <b>Moderate:</b> Distributes the interactive 36-seat visual bus selector as a standalone JS library. | Medium (Package JS seat picker into standalone bundle) | <b>FUTURE EXPANSION</b> |
| <b>Release Binary / Archive</b> | GitHub Releases Artifacts                    | <b>Immediate (100%)</b> | <b>High:</b> Pre-packaged offline zip bundles containing source, database dump, and env templates.   | Very Low (Automated zip bundling on release)           | <b>RECOMMENDED #2</b>   |

## 3. Primary Recommendation: Docker Image on GHCR

### 3.1 Architectural Overview

Publishing to **GitHub Container Registry (GHCR)** turns the application into a first-class cloud container. It creates a dedicated **Packages** tab on your GitHub profile and repository, signaling professional DevOps maturity.



### 3.2 Automated CI/CD Workflow (`.github/workflows/docker-publish.yml`)

Using GitHub Actions, image builds and registry publishing are completely automated on every new release or tag push without requiring manual Docker uploads:

```

name: Build and Publish Docker Package

on:
  release:
    types: [published]
  push:
    tags: [ 'v*.*.*' ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Extract metadata (tags, labels)
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
          tags: |
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=raw,value=latest,enable=${ github.ref_type == 'tag' }

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${ steps.meta.outputs.tags }
          labels: ${ steps.meta.outputs.labels }

```

**Zero Extra Secrets Required:** The workflow utilizes the standard built-in `${ secrets.GITHUB_TOKEN }` with `packages: write` permission. No external API keys or paid Docker Hub subscriptions are necessary.

## 4. Secondary Packages: Modular Decoupling

### 4.1 Composer Package: Core Booking Engine

If you wish to distribute business logic for other PHP developers, you can extract a dedicated namespace (e.g., `Jashan\BusReservation\`) containing:

- **PnrGenerator:** Algorithm generating unique, collision-resistant booking identifiers.
- **SeatMatrixCalculator:** Logic calculating available vs. booked seats for 36-passenger buses.
- **DatabaseDriver:** The SSL-aware MySQLi connector wrapper.

Sample `composer.json` Manifest:

```
{
  "name": "jashan-randhawa/bus-reservation-core",
  "description": "Core booking algorithms, PNR generator and seat matrix engine for bus ticketing",
  "type": "library",
  "license": "MIT",
  "authors": [
    { "name": "Jashan Randhawa", "homepage": "https://github.com/Jashan-randhawa" }
  ],
  "require": {
    "php": ">=7.4",
    "ext-mysqli": "*"
  },
  "autoload": {
    "psr-4": { "BusReservation\\": "src/" }
  }
}
```

### 4.2 npm Package: Interactive Seat Picker Widget

The bus layout with 36 seats, click handlers, and status coloration can be packaged as a lightweight vanilla JavaScript UI widget:

- **Package Identifier:** `@jashan-randhawa/bus-seat-picker`
- **Usage:** Any frontend application can instantiate the seat grid with `new BusSeatPicker('#container', { rows: 9, onSelect: callback })`.

## 5. Step-by-Step Implementation Roadmap

| Phase   | Milestone                 | Key Deliverables                                                                                                                                                                                                                                                            | Timeline           |
|---------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| Phase 1 | GHCR Container Publishing | <ul style="list-style-type: none"><li>- Add <code>.github/workflows/docker-publish.yml</code></li><li>- Trigger build for <code>v2.0.0</code> tag</li><li>- Set package visibility to <b>Public</b></li><li>- Add GitHub Packages badge to <code>README.md</code></li></ul> | 1 Hour (Immediate) |

| Phase          | Milestone                            | Key Deliverables                                                                                                                                                                                                                       | Timeline                |
|----------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| <b>Phase 2</b> | <b>Release Asset Bundles</b>         | <ul style="list-style-type: none"> <li>- Automated CI release job generating <code>bus-reservation-v2.0.0.zip</code></li> <li>- Bundles source code, <code>database/init.sql</code>, and sample <code>.env.example</code></li> </ul>   | <b>1 Day</b>            |
| <b>Phase 3</b> | <b>Composer &amp; npm Decoupling</b> | <ul style="list-style-type: none"> <li>- Extract PNR generator into <code>src/</code> with PSR-4 autoloading</li> <li>- Publish to GitHub Packages for Composer or Packagist</li> <li>- Package seat map JS into npm module</li> </ul> | <b>Future Iteration</b> |

## 6. How Consumers Will Run Your Package

Once published to GHCR, running your platform becomes a one-liner for anyone across the globe:

```
# Pull the official pre-built package from GitHub Container Registry
docker pull ghcr.io/jashan-randhawa/bus-reservation:latest

# Run the container connecting to TiDB Cloud or any MySQL instance
docker run -d -p 8080:80 \
  -e DB_HOST=gateway01.ap-south-1.prod.aws.tidbcloud.com \
  -e DB_PORT=4000 \
  -e DB_USER=your_user \
  -e DB_PASS=your_password \
  -e DB_NAME=test \
  -e DB_SSL=true \
  --name bus-app \
  ghcr.io/jashan-randhawa/bus-reservation:latest
```